

Solutions for BMC105 - Computer Organization & Architecture Exam

SECTION A (2 marks each)

a. Functional units of a digital computer and their interconnections

A digital computer consists of five main functional units:

1. **Input Unit:** Accepts data and instructions from external devices (keyboard, mouse, etc.) and converts them into computer-readable format
2. **Output Unit:** Sends processed results to external devices (monitor, printer, etc.)
3. **Memory Unit:** Stores data and instructions, divided into primary memory (RAM, ROM) and secondary storage
4. **Arithmetic Logic Unit (ALU):** Performs arithmetic operations (addition, subtraction) and logical operations (AND, OR, NOT)
5. **Control Unit:** Coordinates and controls all operations, generates control signals, and manages instruction execution

Interconnections: These units are connected via system buses (data bus, address bus, and control bus). The CPU (containing ALU and Control Unit) fetches instructions from memory, processes data, and communicates with input/output devices through the bus system.

b. Difference between Computer Architecture and Computer Organization

Computer Architecture:

- Refers to the functional behavior and logical design of the computer system
- Deals with what the computer does (functionality, instruction sets, addressing modes)
- Focuses on the programmer's view
- Example: Instruction format, data types, memory addressing

Computer Organization:

- Refers to the operational units and their interconnections
 - Deals with how the computer performs operations (hardware implementation)
 - Focuses on the hardware designer's view
 - Example: Control signals, memory technology, bus structure
-

c. Why Look-ahead Carry Adders are faster than Ripple Carry Adders

In **Ripple Carry Adders**, the carry output from each full adder must propagate to the next stage sequentially. This creates a delay as each stage must wait for the previous carry, making total delay proportional to n (number of bits).

In **Look-ahead Carry Adders**, carries are generated simultaneously using additional logic circuits based on generate (G) and propagate (P) functions. All carries are calculated in parallel, significantly reducing propagation delay. The delay is logarithmic rather than linear, making it much faster for multi-bit addition.

d. When is Bus Arbitration required?

Bus arbitration is required when:

- Multiple devices (masters) need to access the system bus simultaneously
 - There's contention for bus control among processors, DMA controllers, or I/O devices
 - A shared bus architecture is used where only one device can control the bus at a time
 - Priority must be established among competing devices to prevent conflicts and ensure orderly data transfer
-

e. Why WMFC (Wait-for-Memory-Function-Completed) is needed

WMFC is needed because:

- Memory operations take multiple clock cycles to complete
 - The CPU operates faster than memory access times
 - The processor must wait for the memory to complete read/write operations before proceeding
 - It prevents the processor from accessing incomplete or invalid data
 - It synchronizes the CPU with slower memory devices, ensuring data integrity
-

f. Representation of $(-8)_{10}$ and $(+8)_{10}$ in two's complement (5-bit)

For $(+8)_{10}$:

- Binary: 01000
- Two's complement: 01000 (positive numbers remain same)

For $(-8)_{10}$:

- Binary of +8: 01000

- One's complement: 10111
 - Add 1: $10111 + 1 = 11000$
 - Two's complement: 11000
-

g. Define Interrupt and how it is handled

Interrupt: An interrupt is a signal sent to the processor by hardware or software indicating an event that needs immediate attention, temporarily suspending normal program execution.

Handling:

1. Complete current instruction execution
 2. Save program counter (PC) and processor status
 3. Identify interrupt source (polling or vectored)
 4. Disable further interrupts (if needed)
 5. Transfer control to Interrupt Service Routine (ISR)
 6. Execute ISR
 7. Restore saved context
 8. Re-enable interrupts
 9. Return to interrupted program
-

SECTION B (7 marks each - Any 3)

a. Different Addressing Modes with Examples

1. Immediate Addressing: Operand is part of instruction

- Example: `MOV A, #50` (Move value 50 to register A)

2. Direct/Absolute Addressing: Address of operand is given in instruction

- Example: `LOAD R1, 500` (Load content of memory location 500 into R1)

3. Indirect Addressing: Address field contains address of effective address

- Example: `ADD R1, @R2` (R2 contains address of operand)

4. Register Addressing: Operand is in a register

- Example: `MOV R1, R2` (Move content of R2 to R1)

5. Register Indirect: Register contains memory address of operand

- Example: `LOAD R1, (R2)` (R2 points to memory location)

6. Indexed Addressing: Effective address = Index register + Address field

- Example: `LOAD R1, 100(X)` (Effective address = X + 100)

7. Relative Addressing: Effective address = PC + Address field

- Example: `JMP +10` (Jump to PC + 10)

8. Auto-increment/decrement: Register value automatically modified

- Example: `LOAD R1, (R2)+` (Use R2, then increment it)
-

b. Booth's Multiplication Method for $(-9)_{10} \times (-13)_{10}$

Step 1: Convert to binary

- $(-9)_{10} = 10111$ (two's complement in 5 bits)
- $(-13)_{10} = 10011$ (two's complement in 5 bits)

Step 2: Booth's Algorithm

- Multiplicand (M) = 10111
- Multiplier (Q) = 10011
- Initialize: A = 00000, $Q_{-1} = 0$
- Count = 5

Iterations:

Step	A	Q	Q_{-1}	Operation	Action
Initial	00000	10011	0	-	-
1	00000	10011	0	$Q_0Q_{-1}=10$	$A=A-M$, Shift
After subtract	01001	10011	0		
After shift	00100	11001	1		
2	00100	11001	1	$Q_0Q_{-1}=11$	Shift only
After shift	00010	01100	1		
3	00010	01100	1	$Q_0Q_{-1}=01$	$A=A+M$, Shift
After add	11001	01100	1		
After shift	11100	10110	0		
4	11100	10110	0	$Q_0Q_{-1}=10$	$A=A-M$, Shift

After subtract	00101	10110	0	
After shift	00010	11011	0	
5	00010	11011	0	$Q_0Q_{-1}=10 \quad A=A-M, \text{ Shift}$
After subtract	01011	11011	0	
After shift	00101	11101	1	

Result: Product = A,Q = 00101 11101 = 117_{10} ✓

Hardware Diagram: Includes two registers (A, Q), one bit Q_{-1} , adder/subtractor, and control logic.

Flowchart: Shows initialization, loop for checking Q_0Q_{-1} bits, arithmetic shift right, and count decrement until complete.

c. IEEE 754 Single Precision representation of $(128.25)_{10}$

Step 1: Convert to binary

- $128_{10} = 10000000_2$
- $0.25_{10} = 0.01_2$
- $128.25_{10} = 10000000.01_2$

Step 2: Normalize

- $10000000.01_2 = 1.000000001_2 \times 2^7$

Step 3: IEEE 754 format (32-bit)

- Sign bit (S) = 0 (positive)
- Exponent (E) = $7 + 127$ (bias) = $134_{10} = 10000110_2$
- Mantissa (M) = 00000000100000000000000 (23 bits, drop leading 1)

Final representation:

0 | 10000110 | 00000000100000000000000
 S | E | M

Hexadecimal: 0x43004000

d. Instruction Execution in Single-Bus Organization

In single-bus organization, all components (CPU, memory, I/O) share a common bus. Instruction execution involves:

Fetch Phase:

1. $PC \rightarrow MAR$ (Program Counter to Memory Address Register)
2. Memory read signal activated
3. WMFC (wait for memory)
4. $MDR \rightarrow IR$ (Memory Data Register to Instruction Register)
5. PC incremented

Decode Phase: Control unit decodes instruction in IR and determines operation type

Execute Phase (example: ADD R1, R2):

1. $R2 \rightarrow Y$ (operand to temporary register)
2. $R1 + Y \rightarrow Z$ (ALU operation)
3. $Z \rightarrow R1$ (result stored back)

Write-back Phase (if needed):

1. Result $\rightarrow MDR$
2. Destination address $\rightarrow MAR$
3. Memory write signal activated
4. WMFC

The single bus is time-multiplexed, so only one data transfer can occur at a time, which can limit performance.

e. Modes of Data Transfer

I/O Transfer Modes:

(i) Programmed I/O:

- CPU continuously checks I/O device status (polling)
- CPU executes program to transfer data
- CPU waits for device to be ready
- Simple but inefficient (CPU time wasted)
- Steps:
 1. CPU checks status register
 2. If ready, transfer data
 3. Repeat until complete
- Example: `LOOP: TEST Status, Ready; JZ LOOP; IN Data`

(ii) Interrupt-Initiated I/O:

- Device sends interrupt signal when ready

- CPU responds when current instruction completes
- More efficient than programmed I/O
- CPU can perform other tasks while waiting
- Steps:
 1. CPU initiates I/O and continues other work
 2. Device raises interrupt when ready
 3. CPU saves context
 4. ISR handles data transfer
 5. CPU resumes interrupted task

Advantages: Interrupt I/O is more efficient as CPU isn't wasted in polling loops

SECTION C (7 marks each - Attempt any one part)

3(a). Bus Arbitration Methods

Bus arbitration determines which device gets control of the bus when multiple devices request it simultaneously.

1. Daisy Chain (Serial) Priority:

- **Method:** Priority line daisy-chained through devices; bus grant signal passes serially
- **Advantages:** Simple, minimal hardware, easy to add devices
- **Disadvantages:** Lower priority devices may starve, sensitive to failures, slower response

2. Polling (Rotating Priority):

- **Method:** Controller polls each device sequentially using address lines
- **Advantages:** Flexible priority assignment, fair access possible
- **Disadvantages:** Additional time for polling, requires control logic

3. Independent Request (Parallel Arbitration):

- **Method:** Each device has separate request and grant lines to arbiter
 - **Advantages:** Fast response, flexible priority, no device blocking
 - **Disadvantages:** More hardware (wiring), complex for many devices
-

3(b). Program for $X = (A-B)/[C+(D \cdot E)]$

(i) 3-Address Instructions (OP dest, src1, src2):

```
MUL T1, D, E    ; T1 = D * E
ADD T2, C, T1    ; T2 = C + (D * E)
SUB T3, A, B     ; T3 = A - B
```

DIV X, T3, T2 ; $X = (A - B) / [C + (D * E)]$

(ii) 2-Address Instructions (OP dest, src; dest = dest OP src):

MOV T1, D ; $T1 = D$
MUL T1, E ; $T1 = D * E$
MOV T2, C ; $T2 = C$
ADD T2, T1 ; $T2 = C + (D * E)$
MOV T3, A ; $T3 = A$
SUB T3, B ; $T3 = A - B$
DIV T3, T2 ; $T3 = (A - B) / [C + (D * E)]$
MOV X, T3 ; $X = T3$

(iii) 1-Address Instructions (OP addr; accumulator-based):

LOAD D ; $ACC = D$
MUL E ; $ACC = D * E$
STORE T1 ; $T1 = D * E$
LOAD C ; $ACC = C$
ADD T1 ; $ACC = C + (D * E)$
STORE T2 ; $T2 = C + (D * E)$
LOAD A ; $ACC = A$
SUB B ; $ACC = A - B$
DIV T2 ; $ACC = (A - B) / [C + (D * E)]$
STORE X ; $X = ACC$

4(a). Division: 8/3 using Restoring Division Method

Dividend (Q): $8_{10} = 1000_2$ (4 bits) **Divisor (M):** $3_{10} = 0011_2$

Setup:

- A = 0000 (initially)
- Q = 1000
- M = 0011
- n = 4 iterations

Step	A	Q	Operation	Result
Initial	0000	1000	-	-
1. Shift left	0001	0000	AQ << 1	
Subtract M	1110	0000	A - M (negative)	
Restore	0001	0000	A = A + M, $Q_0 = 0$	

2. Shift left	0010	0000	AQ << 1
Subtract M	1111	0000	A - M (negative)
Restore	0010	0000	A = A + M, Q ₀ = 0
3. Shift left	0100	0000	AQ << 1
Subtract M	0001	0000	A - M (positive)
Keep	0001	0001	Q ₀ = 1
4. Shift left	0010	0010	AQ << 1
Subtract M	1111	0010	A - M (negative)
Restore	0010	0010	A = A + M, Q ₀ = 0

Result:

- Quotient (Q) = $0010_2 = 2_{10}$
- Remainder (A) = $0010_2 = 2_{10}$
- Verification: $8 = 3 \times 2 + 2$ ✓

4(b). Flowchart for Floating Point Multiplication

Steps in floating point multiplication:

1. **Start**
2. **Check for zeros:** If either operand is zero, result is zero
3. **Add exponents:** $E = E_1 + E_2 - \text{Bias}$
4. **Multiply mantissas:** $M = M_1 \times M_2$
5. **Normalize result:**
 - If $M \geq 2$, shift right and increment E
 - If $M < 1$, shift left and decrement E
6. **Check exponent overflow/underflow:**
 - If $E > \text{Max}$, overflow
 - If $E < \text{Min}$, underflow
7. **Determine sign:** $\text{Sign} = \text{Sign}_1 \text{ XOR } \text{Sign}_2$
8. **Round mantissa** (if needed)
9. **Store result**
10. **End**

Flowchart elements include: Decision diamonds for zero checks, overflow/underflow checks; process boxes for add, multiply, normalize operations; and proper flow arrows showing the sequence.

5(a). Arithmetic Pipeline and Instruction Pipeline

Arithmetic Pipeline:

- Divides arithmetic operations into sub-operations
- Each sub-operation performed in different pipeline stage
- Example: Floating point addition pipeline
 - Stage 1: Compare exponents
 - Stage 2: Align mantissas
 - Stage 3: Add mantissas
 - Stage 4: Normalize result
- **Diagram:** Shows sequential stages with registers between them
- **Advantage:** Multiple operations in progress simultaneously
- **Throughput:** One result per clock cycle after pipeline fills

Instruction Pipeline:

- Overlaps execution of multiple instructions
 - Typical stages:
 - IF (Instruction Fetch)
 - ID (Instruction Decode)
 - EX (Execute)
 - MEM (Memory Access)
 - WB (Write Back)
 - **Diagram:** Shows 5-stage pipeline with different instructions at different stages
 - **Hazards:** Structural, data, control hazards can stall pipeline
 - **Speedup:** Ideally n times for n stages
-

5(b). Differences between RISC and CISC

Feature	RISC	CISC
Instruction Set	Small, simple	Large, complex
Instruction Format	Fixed length	Variable length
Addressing Modes	Few (2-3)	Many (10+)
Execution Time	One cycle per instruction	Multiple cycles
Programming	More instructions needed	Fewer instructions
Registers	Many general purpose	Few registers
Pipeline	Easily pipelined	Difficult to pipeline
Compiler	Complex compiler	Simple compiler

Memory Access	Load/Store only	Any instruction
Instruction Complexity	Simple	Complex
Examples	ARM, MIPS, SPARC	Intel x86, VAX
Design Philosophy	Hardware simplicity	Hardware complexity
Code Size	Larger	Smaller
Power Consumption	Lower	Higher

6(a). Virtual Memory Page Replacement

Given:

- Address space: 8K words
- Memory space: 4K words
- Page/Frame size: 1K words
- Number of frames: $4K/1K = 4$ frames
- Page reference string: 4, 2, 0, 1, 2, 6, 1, 4, 0, 1, 0, 2, 3, 5, 7

(a) FIFO (First In First Out):

Reference	Frame 0	Frame 1	Frame 2	Frame 3	Page Fault
4	4	-	-	-	✓
2	4	2	-	-	✓
0	4	2	0	-	✓
1	4	2	0	1	✓
2	4	2	0	1	-
6	6	2	0	1	✓
1	6	2	0	1	-
4	6	4	0	1	✓
0	6	4	0	1	-
1	6	4	0	1	-
0	6	4	0	1	-

2	6	4	2	1	✓
3	6	4	2	3	✓
5	5	4	2	3	✓
7	5	7	2	3	✓

Total Page Faults (FIFO): 10

(b) LRU (Least Recently Used):

Reference	Frame 0	Frame 1	Frame 2	Frame 3	Page Fault
4	4	-	-	-	✓
2	4	2	-	-	✓
0	4	2	0	-	✓
1	4	2	0	1	✓
2	4	2	0	1	-
6	6	2	0	1	✓
1	6	2	0	1	-
4	6	2	4	1	✓
0	6	0	4	1	✓
1	6	0	4	1	-
0	6	0	4	1	-
2	2	0	4	1	✓
3	2	0	3	1	✓
5	2	0	3	5	✓
7	2	7	3	5	✓

Total Page Faults (LRU): 11

6(b). 2D vs 2.5D Memory Organization

2D Memory Organization:

- Traditional organization with rows and columns
- Single bank of memory cells arranged in matrix
- Access: Row decoder + Column decoder
- **Diagram:** Shows single memory array with row/column decoders
- **Characteristics:**
 - Simple structure
 - One access at a time per bank
 - Lower cost
 - Sequential access to different rows causes delay

2.5D Memory Organization (Multi-bank):

- Memory divided into multiple banks
- Each bank has independent row/column decoders
- Can access multiple banks simultaneously
- **Diagram:** Shows multiple independent banks with shared address bus
- **Characteristics:**
 - Parallel access to different banks
 - Higher throughput
 - Bank conflicts can occur
 - Used in modern DRAM (DDR)
 - Better performance for interleaved access patterns

Key Differences:

1. **Parallelism:** 2.5D allows concurrent bank operations
 2. **Performance:** 2.5D offers higher bandwidth
 3. **Complexity:** 2.5D requires more control logic
 4. **Cost:** 2.5D is more expensive
 5. **Applications:** 2.5D used in high-performance systems
-

7(a). Asynchronous Data Transfer

Asynchronous Data Transfer: Transfer where data exchange occurs without a common clock signal between sender and receiver. Timing is controlled by handshaking signals.

Methods:

1. Strobe Control:

- Uses single control signal (strobe) to indicate data validity
- **Source-Initiated:** Source places data and activates strobe
 - Simple, fast

- No acknowledgment from destination
- **Destination-Initiated:** Destination requests data with strobe
 - Destination controls timing
 - Source responds to strobe

2. Handshaking (Interlocked Protocol):

- Uses two control signals for acknowledgment
- More reliable than strobe
- Two types:

a) Source-Initiated Handshaking:

1. Source places data on bus
2. Source sets "Data Ready" signal
3. Destination reads data
4. Destination sets "Data Accepted" signal
5. Source removes data and "Data Ready"
6. Destination removes "Data Accepted"

b) Destination-Initiated Handshaking:

1. Destination sets "Ready for Data"
2. Source places data and sets "Data Valid"
3. Destination reads data, removes "Ready for Data"
4. Source removes data and "Data Valid"

Advantages of Handshaking:

- Reliable synchronization
- Works with devices of different speeds
- Error detection possible
- More robust than strobe

Timing Diagrams: Show signal transitions for both methods

7(b). DMA Controller

DMA (Direct Memory Access) Controller Block Diagram:

Components:

1. **Address Register:** Holds memory address for transfer
2. **Word Count Register:** Counts number of words to transfer
3. **Control Register:** Stores transfer mode and control information
4. **Data Register:** Temporary storage for data being transferred
5. **Control Logic:** Manages DMA operations and bus arbitration
6. **Bus Interface:** Connects to system bus

Why Read and Write Control Lines are Bidirectional:

1. **CPU to Memory Transfers:**
 - DMA reads from I/O device
 - DMA writes to memory
 - Read line: INPUT (from CPU perspective)
 - Write line: OUTPUT (to memory)
2. **Memory to CPU Transfers:**
 - DMA reads from memory
 - DMA writes to I/O device
 - Read line: OUTPUT (to memory)
 - Write line: INPUT (from CPU perspective)
3. **Flexibility:** DMA controller acts as both master and slave
 - As bus master: Initiates read/write to memory
 - Configuration phase: CPU writes to DMA registers
4. **Modes:** Supports both input and output operations
 - Input: Peripheral → Memory (DMA reads from device, writes to memory)
 - Output: Memory → Peripheral (DMA reads from memory, writes to device)

Operational Steps:

1. CPU initializes DMA (address, count, direction)
2. I/O device requests DMA service
3. DMA requests bus from CPU
4. CPU grants bus (enters hold state)
5. DMA transfers data directly between memory and I/O
6. DMA releases bus after transfer complete
7. DMA interrupts CPU to signal completion

Advantages: High-speed transfer, frees CPU for other tasks, efficient for bulk data transfers.